

CS231n 13-15

김동원

목차

Self Supervised Learning

VAE, Variational Autoencoder

GAN

Reinforcement Learning

Efficient method

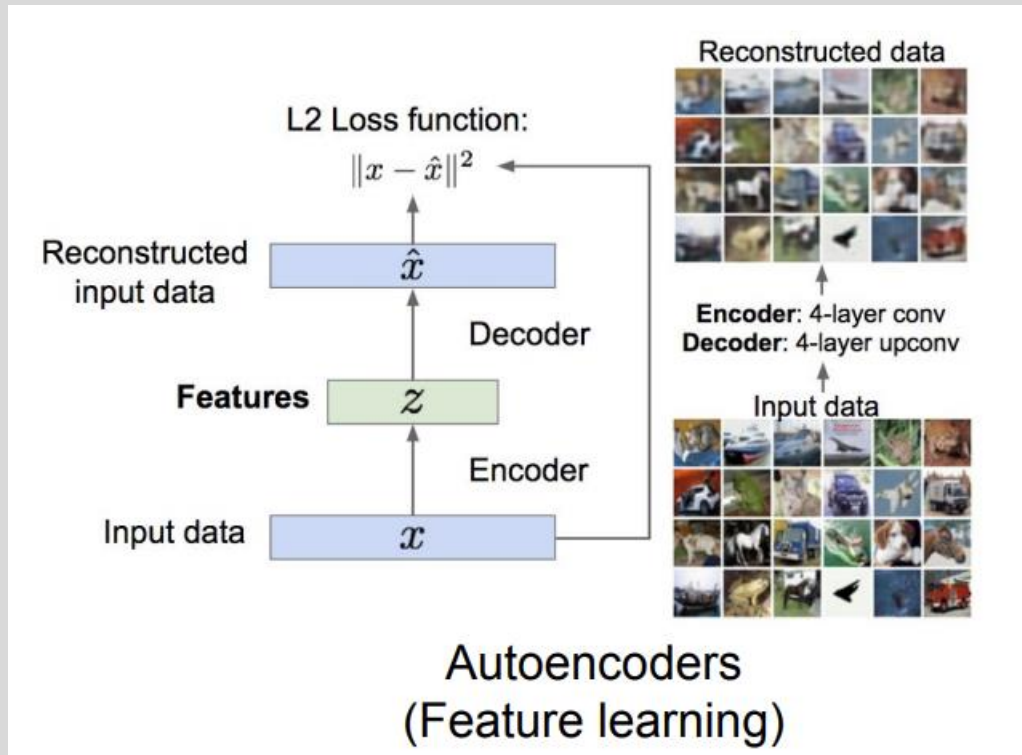
GAN 코드 구현

CNN의 원리

Self Supervised Learning

라벨이 없는 데이터를 학습한다.

Auto encoder



Autoencoder데이터의 특징을 잘 추출하기 위한 방법으로

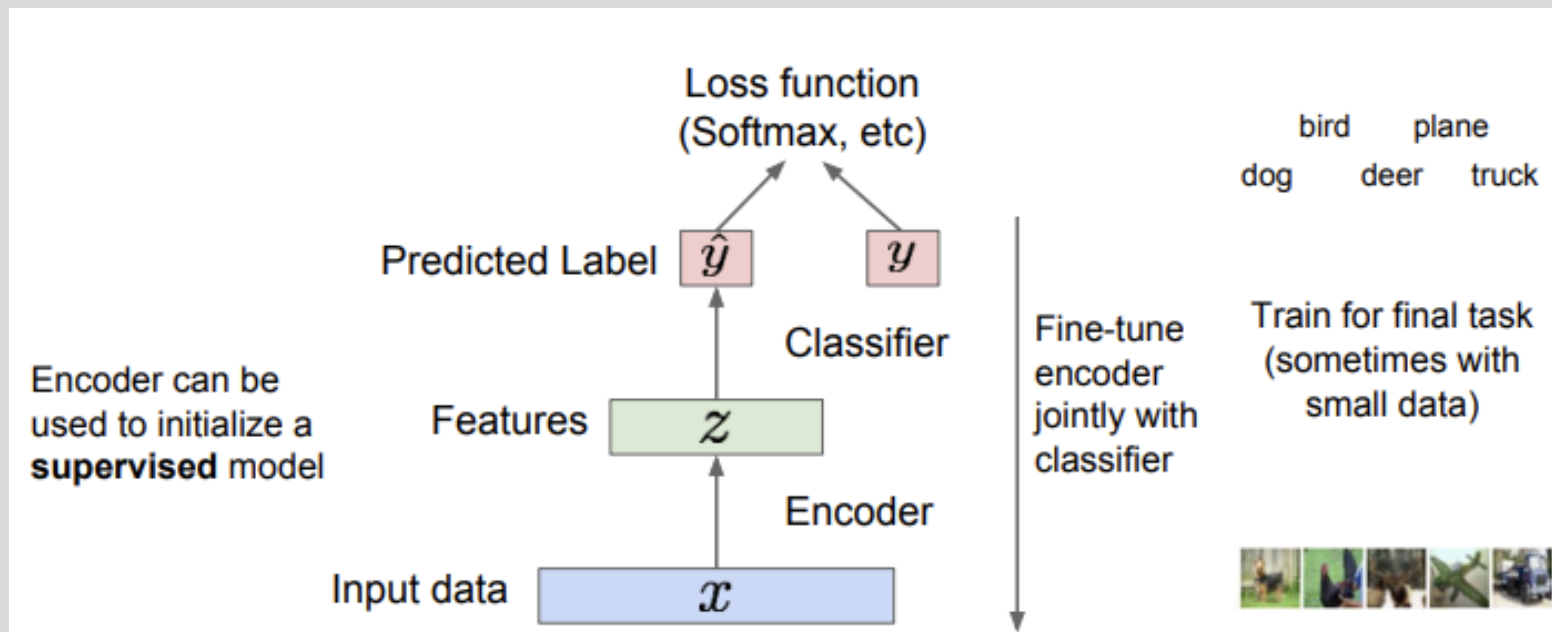
encoder는 입력 데이터를 압축하고 요약한다
Decoder는 encoder를 통과한 데이터를 통해 입력과 비슷하게 재구성한다.

이미지 특징 추출에 대해 학습시키고

Self Supervised Learning

라벨이 없는 데이터를 학습한다.

Auto encoder

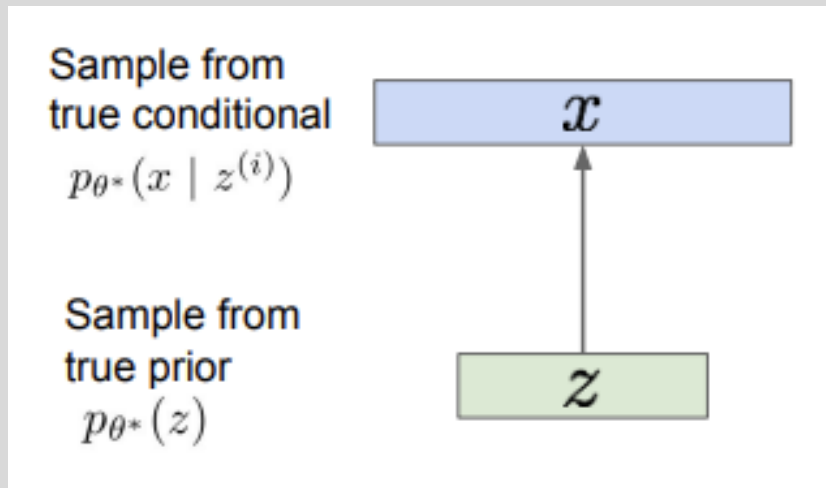


Decoder를 없애고 encoder를 거친 데이터를 분류하는 것을 학습시킨다.
Encoder를 사전 학습된 지도학습 모델의 가중치로 초기화 시킬 수 있다.

VAE, Variational Autoencoder

오토인코더에서 특징을 잘 추출해 내는 것을 보고 새로운 이미지를 생성하기 위해 변형시킨 모델

Autoencoder에서는 encoder를 학습시키기 위해 decode를 사용하였지만 VAE는 decoder를 통해 새로운 이미지 생성을 위해 encoder를 사용한다.



$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

Q: What is the problem with this?

Intractable!

Z의 경우도 encoder의 값에서 랜덤하게 샘플링 된 값인데 P(x)를 구하기위해 모든 z의 값을 알아야 되기 때문에 이는 계산하기 어렵다.

압축된 특징을 나타내는 벡터 z 를 통해 원본 이미지 x 가 가진 특징들의 분포를 예측하는 것이다.
따라서 원하는 것은 $P(x)$ 인데

VAE, Variational Autoencoder

Data likelihood: $p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$

Posterior density also intractable: $p_{\theta}(z|x) = p_{\theta}(x|z) p_{\theta}(z) / p_{\theta}(x)$

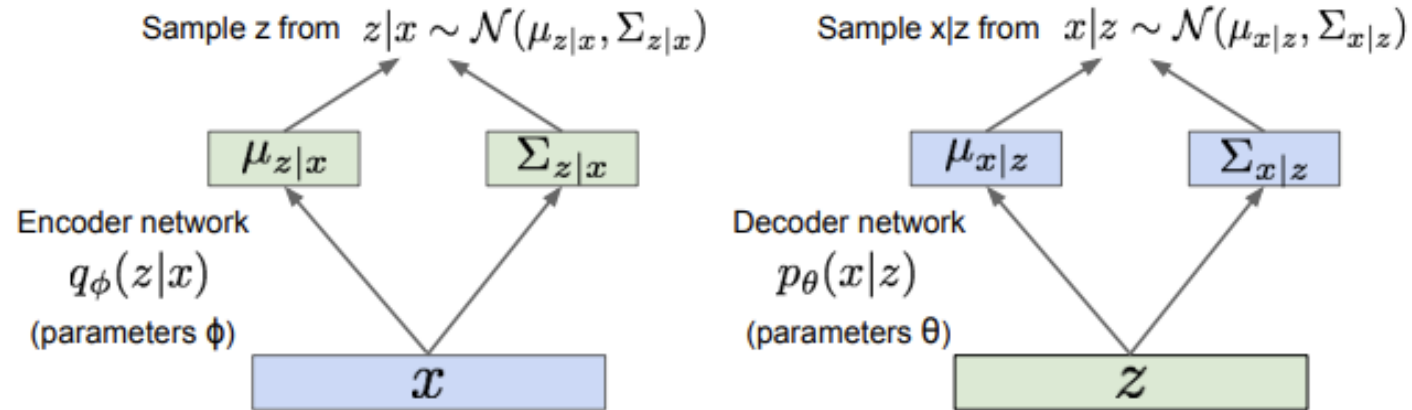
Solution: In addition to decoder network modeling $p_{\theta}(x|z)$, define additional encoder network $q_{\phi}(z|x)$ that approximates $p_{\theta}(z|x)$

Will see that this allows us to derive a lower bound on the data likelihood that is tractable, which we can optimize

Encoder에 $q(z|x)$ 의 네트워크를 추가하여 $p(z|x)$ 에 근접한 대략적인 값을 알 수 있다

VAE, Variational Autoencoder

Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic



Encoder and decoder networks also called
'recognition'/'inference' and 'generation' networks

Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Encoder에 $q(z|x)$ 의 네트워크를 추가하여 $P(z|x)$ 에 근접한 대략적인 값을 알 수 있다

VAE, Variational Autoencoder

Now equipped with our encoder and decoder networks, let's work out the (log) data likelihood:

$$\begin{aligned}
 \log p_{\theta}(x^{(i)}) &= \mathbf{E}_{z \sim q_{\phi}(z|x^{(i)})} \left[\log p_{\theta}(x^{(i)}) \right] && (p_{\theta}(x^{(i)}) \text{ Does not depend on } z) \\
 &= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)} | z) p_{\theta}(z)}{p_{\theta}(z | x^{(i)})} \right] && (\text{Bayes' Rule}) \\
 &= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)} | z) p_{\theta}(z)}{p_{\theta}(z | x^{(i)})} \frac{q_{\phi}(z | x^{(i)})}{q_{\phi}(z | x^{(i)})} \right] && (\text{Multiply by constant}) \\
 &= \mathbf{E}_z \left[\log p_{\theta}(x^{(i)} | z) \right] - \mathbf{E}_z \left[\log \frac{q_{\phi}(z | x^{(i)})}{p_{\theta}(z)} \right] + \mathbf{E}_z \left[\log \frac{q_{\phi}(z | x^{(i)})}{p_{\theta}(z | x^{(i)})} \right] && (\text{Logarithms}) \\
 &= \mathbf{E}_z \left[\log p_{\theta}(x^{(i)} | z) \right] - D_{KL}(q_{\phi}(z | x^{(i)}) || p_{\theta}(z)) + D_{KL}(q_{\phi}(z | x^{(i)}) || p_{\theta}(z | x^{(i)}))
 \end{aligned}$$

Decoder network gives $p_{\theta}(x|z)$, can compute estimate of this term through sampling. (Sampling differentiable through reparam. trick see paper)

This KL term (between Gaussians for encoder and z prior) has nice closed-form solution!

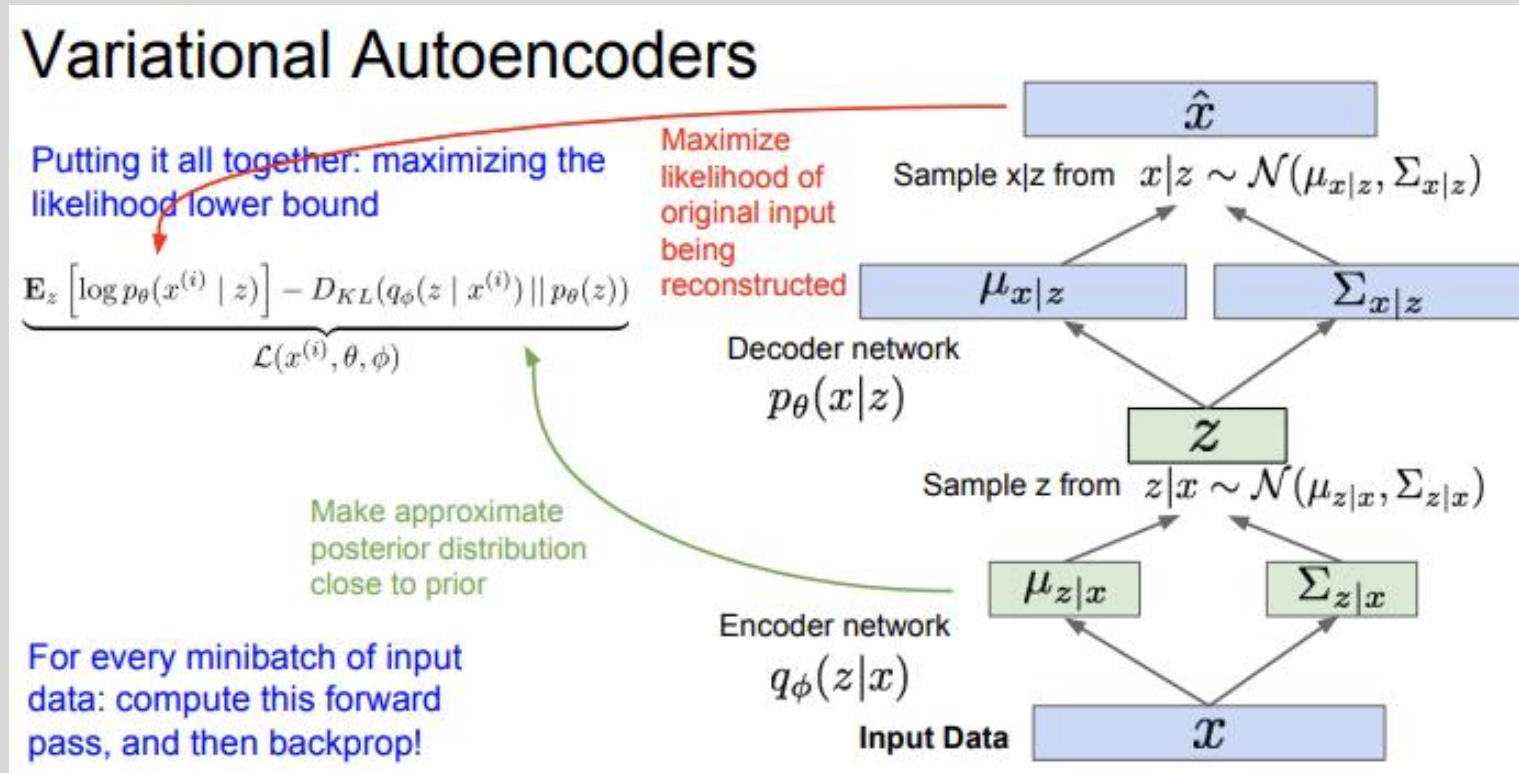
$p_{\theta}(z|x)$ intractable (saw earlier), can't compute this KL term :(But we know KL divergence always ≥ 0 .

$$= \underbrace{\mathbf{E}_z \left[\log p_{\theta}(x^{(i)} | z) \right] - D_{KL}(q_{\phi}(z | x^{(i)}) || p_{\theta}(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)} + \underbrace{D_{KL}(q_{\phi}(z | x^{(i)}) || p_{\theta}(z | x^{(i)}))}_{\geq 0}$$

Tractable lower bound which we can take gradient of and optimize! ($p_{\theta}(x|z)$ differentiable, KL term differentiable)

Z가 $q(z|x)$ 를 따른다는 것을 통해 $p(x)$ 를 구할 수 있게 되는데
 이 값은 = (디코더의 값) - (인코더로 추출한 z와 가정한 z의 차이) + (계산은 불가능하지만 양수인 것은 아는 값)

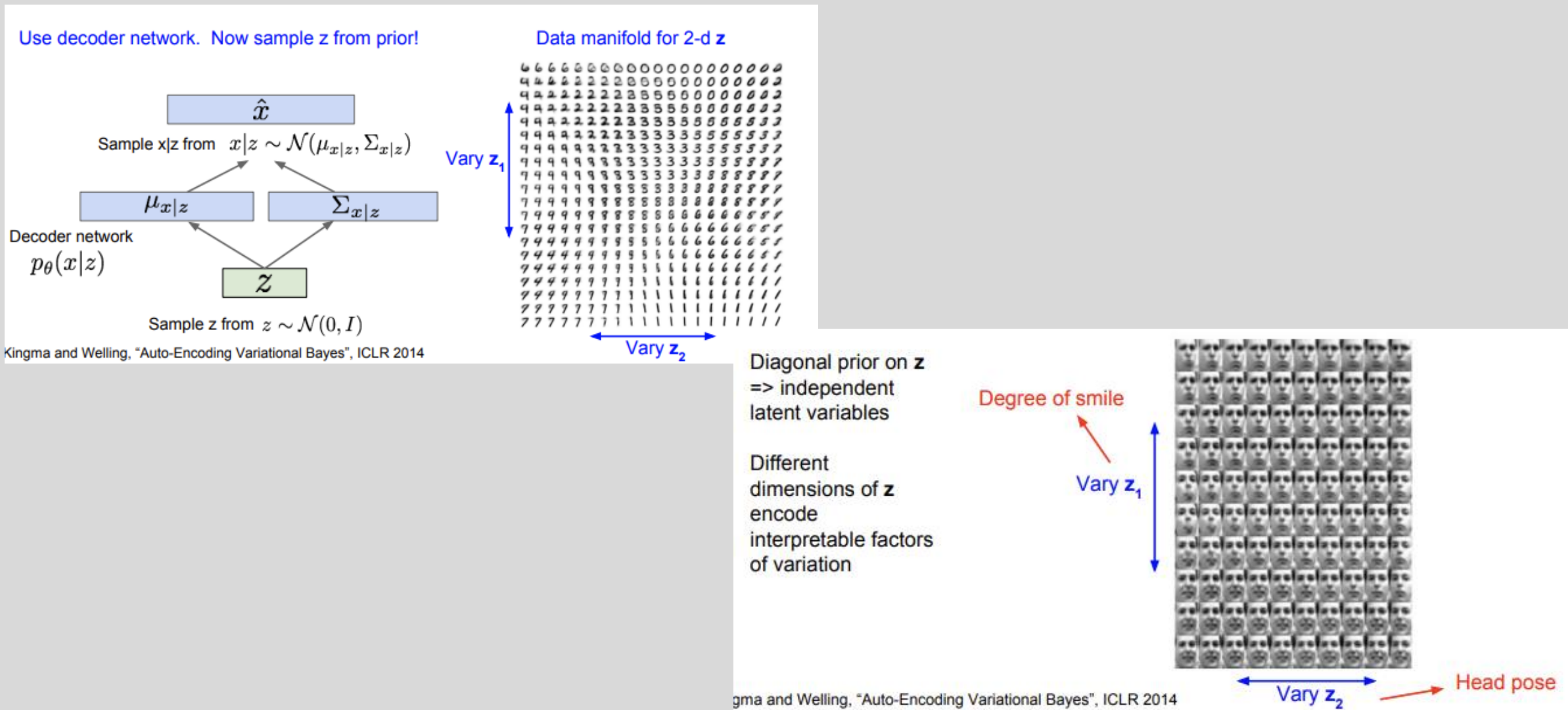
VAE, Variational Autoencoder



결과적으로 Loss는 다음과 같고 이미지를 생성하기 위해서는 decoder를 이용하게 된다.

VAE, Variational Autoencoder

디코더를 통해 이미지를 생성하게 되는데 z 의 샘플에 따라 이미지가 달라지는 것을 볼 수 있다.

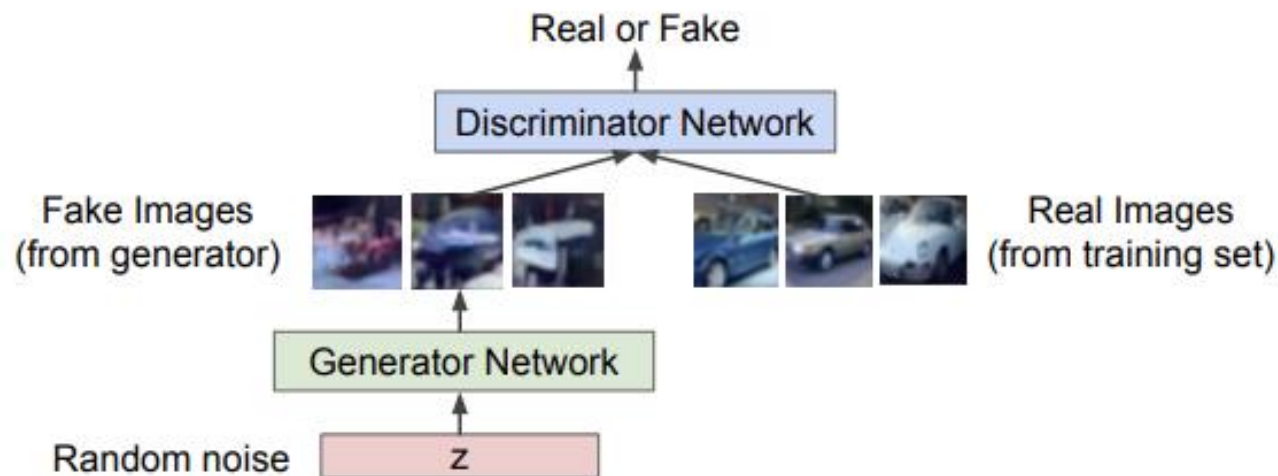


GAN

VAE는 확률밀도함수에 따라 학습을 시켰는데, GAN은 게임이론을 따라 2개의 플레이어를 통해 학습데이터로부터 생성하는 방법이다.

Generator network: try to fool the discriminator by generating real-looking images

Discriminator network: try to distinguish between real and fake images



Train jointly in **minimax game**

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log \underbrace{D_{\theta_d}(x)}_{\text{Discriminator output for real data } x} + \mathbb{E}_{z \sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\text{Discriminator output for generated fake data } G(z)}) \right]$$

Generator와 discriminator로 구성되고

generator는 나타낼 이미지의 분포가 매우 복잡하기 때문에 이를 직접 반환하기 어려워 간단한 분포를 변형하는 방법으로 학습을 진행하고,

Discriminator는 generator의 이미지와 원본 이미지를 구별하는 역할을 하여

Discriminator는 원본 이미지에 대해서 1의 값을 반환하고 generator의 이미지에 0을 반환하여 목적함수를 최대화 하고, generator의 경우는 생성된 이미지를 discriminator에 주어졌을 때 0이 반환되도록 하여 목적함수를 최소화 시키는 방향으로 학습한다.

GAN

MinMax학습의 경우 generator의 학습이 잘 되지 않는다

Alternate between:

1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

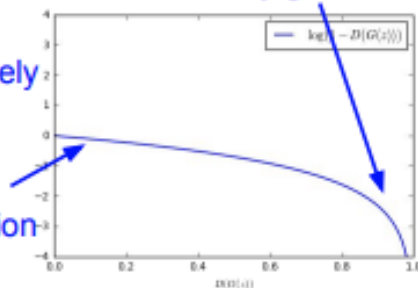
Gradient signal dominated by region where sample is already good

2. **Gradient descent** on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

In practice, optimizing this generator objective does not work well!

When sample is likely fake, want to learn from it to improve generator. But gradient in this region is relatively flat!



X축은 $D(G(x))$ 로 1인 경우 생성된 이미지가 진짜와 같다는 것이고, 0인 경우 생성된 이미지가 좋지 못하다는 의미인데 이때의 기울기는 0에 가까워 생성 이미지가 좋지 못할 경우에는 상당한 학습을 시켜야 된다는 것을 의미한다.

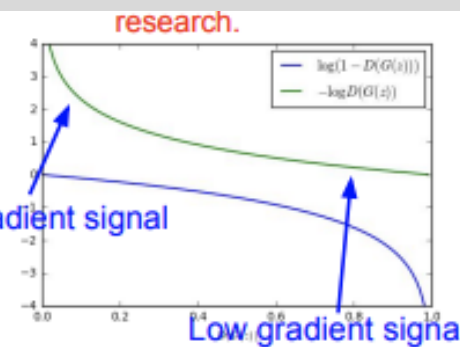
2. **Instead: Gradient ascent** on generator, **different objective**

$$\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$$

Instead of minimizing likelihood of discriminator being correct, now maximize likelihood of discriminator being wrong. Same objective of fooling discriminator, but now higher gradient signal for bad samples => works much better! Standard in practice.

High gradient signal

Low gradient signal



따라서 generator를 Discriminator가 잘못 판단한 경우를 최대화하는 방법으로 목적함수를 설정

GAN

GAN의 학습 순서는 discriminator를 조금 학습시키고 그 다음 generator를 학습시킨다.

Putting it together: GAN training algorithm

Some find $k=1$ more stable, others use $k > 1$, no best rule.

Recent work (e.g. Wasserstein GAN) alleviates this problem, better stability!

```
for number of training iterations do
  for  $k$  steps do
    • Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
    • Sample minibatch of  $m$  examples  $\{x^{(1)}, \dots, x^{(m)}\}$  from data generating distribution  $p_{\text{data}}(x)$ .
    • Update the discriminator by ascending its stochastic gradient:
      
$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D_{\theta_d}(x^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(z^{(i)})))]$$

    end for
    • Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
    • Update the generator by ascending its stochastic gradient (improved objective):
      
$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(D_{\theta_d}(G_{\theta_g}(z^{(i)})))$$

  end for
end for
```

학습 이후에는 generator를 이용하여 이미지를 생성하게 된다.

Generator의 경우 초기에는 fc로 구성되어 성능이 떨어졌지만 이후 DCGAN에서는 CNN을 통해 upsampling하여 공간적 특성을 반영하여 더 좋은 이미지를 얻게 되었다.

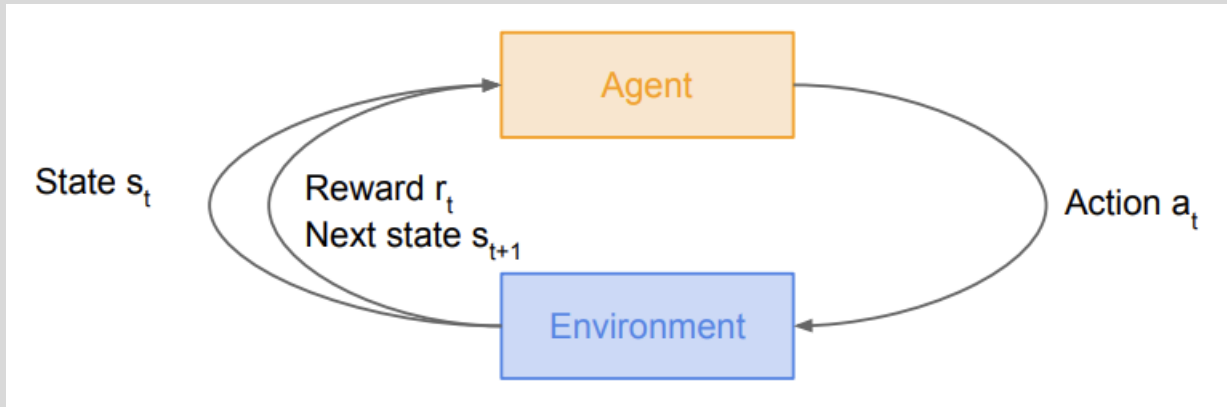
우선 K번 discriminator를 학습시키는데, 이때 랜덤한 노이즈에서 m개를 generator를 통과시키고, m개를 원본 이미지에서 추출하여 학습시킨다.

어느 정도의 학습이 진행되고, generator를 학습하여 초기의 랜덤 노이즈를 discriminator가 속이려는 방향으로 변형시키게 될 것이다.

이 과정을 반복하여 학습을 진행한다.

Reinforcement Learning

보상을 통해 특정행동을 하도록 학습시킨다.



상태를 주면 agent가 행동을 수행하고 이것의 보상과 다음 상태를 agent에게 준다

강화학습 예시

Atari Games



Objective: Complete the game with the highest score

State: Raw pixel inputs of the game state

Action: Game controls e.g. Left, Right, Up, Down

Reward: Score increase/decrease at each time step

Atari 게임을 강화학습으로 모델링할 경우
목표는 게임에서 높은 점수를 획득하는 것이고,
상태는 게임의 상태에 대한 정보이고,
행동은 위, 아래, 왼쪽, 오른쪽으로 움직이는 것이고,
보상의 매 행동에 따른 점수의 증가와 감소를 의미한다.

Reinforcement Learning

Markov Decision Process로 수학적으로 표현할 수 있다.

Defined by: $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathbb{P}, \gamma)$

$\mathcal{S}$: set of possible states

$\mathcal{A}$: set of possible actions

$\mathcal{R}$: distribution of reward given (state, action) pair

$\mathbb{P}$: transition probability i.e. distribution over next state given (state, action) pair

γ : discount factor

- At time step $t=0$, environment samples initial state $s_0 \sim p(s_0)$
- Then, for $t=0$ until done:
 - Agent selects action a_t
 - Environment samples reward $r_t \sim R(\cdot | s_t, a_t)$
 - Environment samples next state $s_{t+1} \sim P(\cdot | s_t, a_t)$
 - Agent receives reward r_t and next state s_{t+1}
- A policy π is a function from S to A that specifies what action to take in each state
- **Objective:** find policy π^* that maximizes cumulative discounted reward: $\sum_{t \geq 0} \gamma^t r_t$

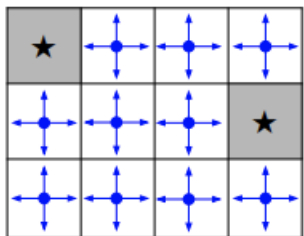
S 는 상태, A 는 행동, R 은 상태와 행동에 대한 보상, P 는 상태 행동에 따라 다음 상태에 전이될 확률, γ 는 미래 보상의 중요도를 나타낸다.

S_{t+1} 은 s_t 와 a_t 쌍에서 P 에 따라 결정된다.

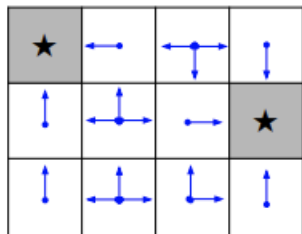
정책은 각 상태에서 어떠한 행동을 할 것인지에 대한 함수이고, π^* 라는 리워드를 최대화 하는 정책을 찾는 것이 목적이다

Reinforcement Learning

정책을 이해하기 위한 예시



Random Policy



Optimal Policy

How do we handle the randomness (initial state, transition probability...)?

Maximize the **expected sum of rewards!**

Formally: $\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi \right]$ with $s_0 \sim p(s_0), a_t \sim \pi(\cdot \mid s_t), s_{t+1} \sim p(\cdot \mid s_t, a_t)$

π^* 는 보상을 최대화 하는 것인데 MDP에서 초기 상태가 랜덤하고, P의 경우 확률적이기 때문에 무작위성이 있어 보상합에 대한 기댓값을 최대화 시킨다.

Reinforcement Learning

정책을 최적화 하기위해 가치함수(value function)과 Q-가치 함수(Q-value function)이 사용되는데

How good is a state?

The **value function** at state s , is the expected cumulative reward from following the policy from state s :

$$V^\pi(s) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | s_0 = s, \pi \right]$$

How good is a state-action pair?

The **Q-value function** at state s and action a , is the expected cumulative reward from taking action a in state s and then following the policy:

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | s_0 = s, a_0 = a, \pi \right]$$

정책은 상태에 따라 행동하는 방식에 대한 것이므로 하나의 상태를 알면 정책에 따라 계속 상태와 행동 보상의 그 경로를 알 수 있게 되는데, 가치함수는 그에 따라 행동에 대한 보상의 기대값이고, Q 가치 함수는 행동과 상태에 따른 보상의 기대값이다.

Reinforcement Learning

Bellman 방정식 -> Bellman 방정식은 어느 시점 t 에서 $t+1$ 에서의 가치함수와 정책의 관계를 다룬다

The optimal Q-value function Q^* is the maximum expected cumulative reward achievable from a given (state, action) pair:

$$Q^*(s, a) = \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi \right]$$

Q^* satisfies the following **Bellman equation**:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

Intuition: if the optimal state-action values for the next time-step $Q^*(s', a')$ are known, then the optimal strategy is to take the action that maximizes the expected value of $r + \gamma Q^*(s', a')$

Q^* 는 최적의 정책이 주어졌을 때 최대의 보상을 얻을 수 있는 상태와 행동의 쌍을 찾을 것이고, 주어진 상태 s 와 a 에 따라 기대 보상은 a 에 따른 보상 r 과 s 상태 이후의 s', a' 일 때의 Q^* 값에 γ 를 곱한 값이 된다

Reinforcement Learning

최적의 정책을 얻기 위해 value iteration 알고리즘 사용하여 모든 상황에서 모든 행동을 비교하여 업데이트하는 방법

Value iteration algorithm: Use Bellman equation as an iterative update

$$Q_{i+1}(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q_i(s', a') | s, a \right]$$

Q_i will converge to Q^* as $i \rightarrow \infty$

What's the problem with this?

Not scalable. Must compute $Q(s, a)$ for every state-action pair. If state is e.g. current game state pixels, computationally infeasible to compute for entire state space!

Solution: use a function approximator to estimate $Q(s, a)$. E.g. a neural network!

하지만 이 방법은 많은 상태와 행동에 대해 q 가치를 계산해야 되어 실현 불가능하다. 따라서 대략적인 값을 예측할 수 있는 신경망을 사용한다.

Reinforcement Learning

Q-learning은 Q 가치 함수를 업데이트 하는 방법이다.

Q-learning: Use a function approximator to estimate the action-value function

$$Q(s, a; \theta) \approx Q^*(s, a)$$

function parameters (weights)

Remember: want to find a Q-function that satisfies the Bellman Equation:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') | s, a \right]$$

Forward Pass

Loss function: $L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot)} [(y_i - Q(s, a; \theta_i))^2]$

where $y_i = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) | s, a \right]$

Iteratively try to make the Q-value close to the target value (y_i) it should have, if Q-function corresponds to optimal Q^* (and optimal policy π^*)

Backward Pass

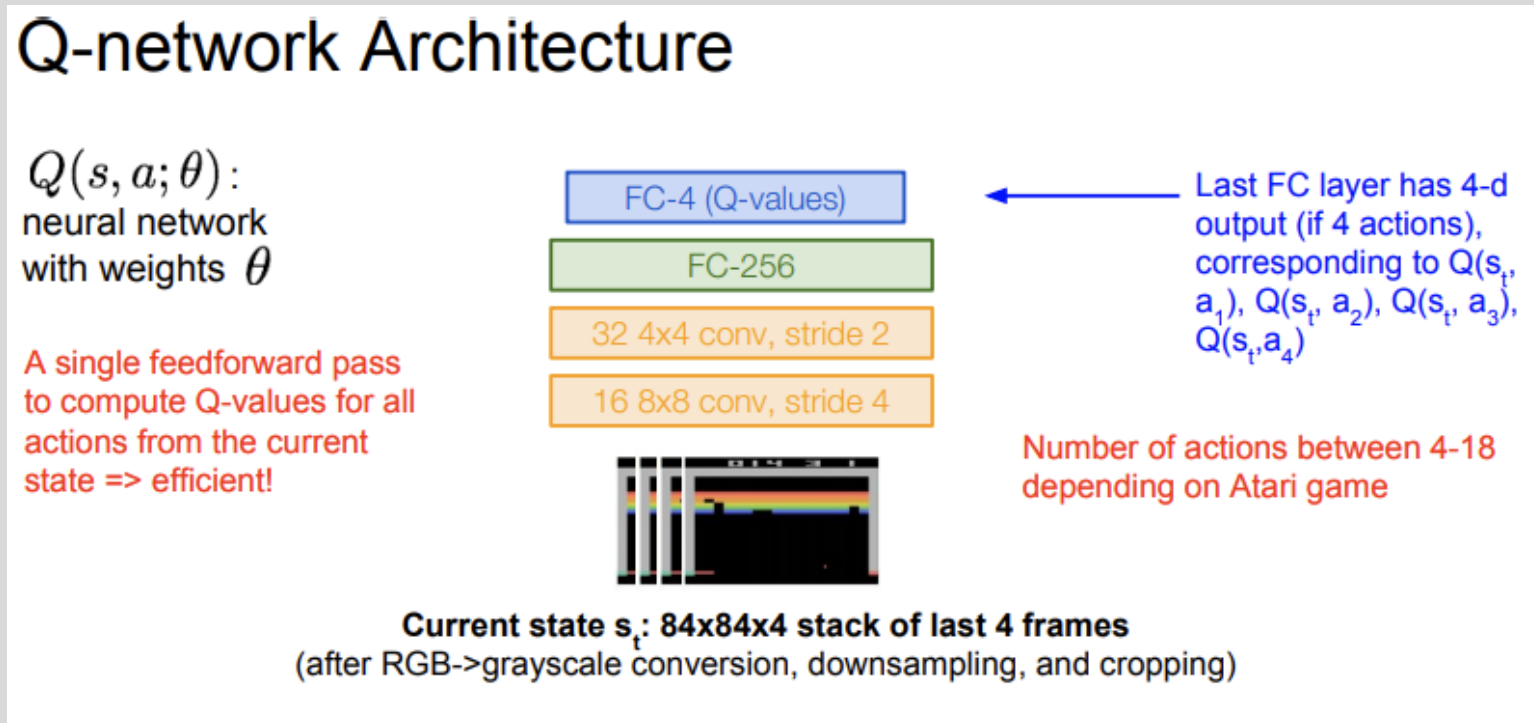
Gradient update (with respect to Q-function parameters θ):

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot); s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i) \right] \nabla_{\theta_i} Q(s, a; \theta_i)$$

Loss는 $q(s,a)$ 가 목적함수와의 거리를 측정하고, 이때 목적함수는 bellman방정식을 의미한다. Forward 과정에서는 loss를 최소화 시키고, backward 과정에서는 손실함수를 세타에 대해 미분하여 업데이트 시키는 방법이다. Q함수를 갱신 시키는 방법. Q value regression과 같다.

Reinforcement Learning

Deep Q network는 예시



상태는 게임의 화면이 되고, Fc의 출력이 각 상태의 행동이고, 한번의 forward 과정으로 한 상태에서 하나의 action의 reward를 반영하여 Q value 계산 가능하여 효율적이다

Reinforcement Learning

Q network를 학습할 경우의 문제 중에 experience replay라는 것이 존재한다

게임 화면과 같은 샘플을 통해 상태 행동 보상을 결정하는데, 모든 샘플들이 상관 관계를 가져 학습에 비효율적이라는 것이다. 또한 q network의 파라미터를 보았을 때 어떤 행동을 할 것 인지의 정책을 정하는 것이 어떤 샘플을 원하는 지를 결정하는 것과 같다는 것이고, 이는 학습에 좋지 않은 영향을 미친다.

이를 해결하기 위해 replay memory에 전이되는(P) (상태, 행동, 보상, 다음 상태)로 구성된 테이블로 명시하고 mini batch의 replay memory를 통해 학습을 시키고 많은 플레이를 통해 replay memory를 업데이트 한다.

Reinforcement Learning

Q network를 학습할 경우의 문제 중에 experience replay라는 것이 존재한다

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N

Initialize action-value function Q with random weights

for episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$

for $t = 1, T$ **do**

 With probability ϵ select a random action a_t

 otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$

 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3

end for

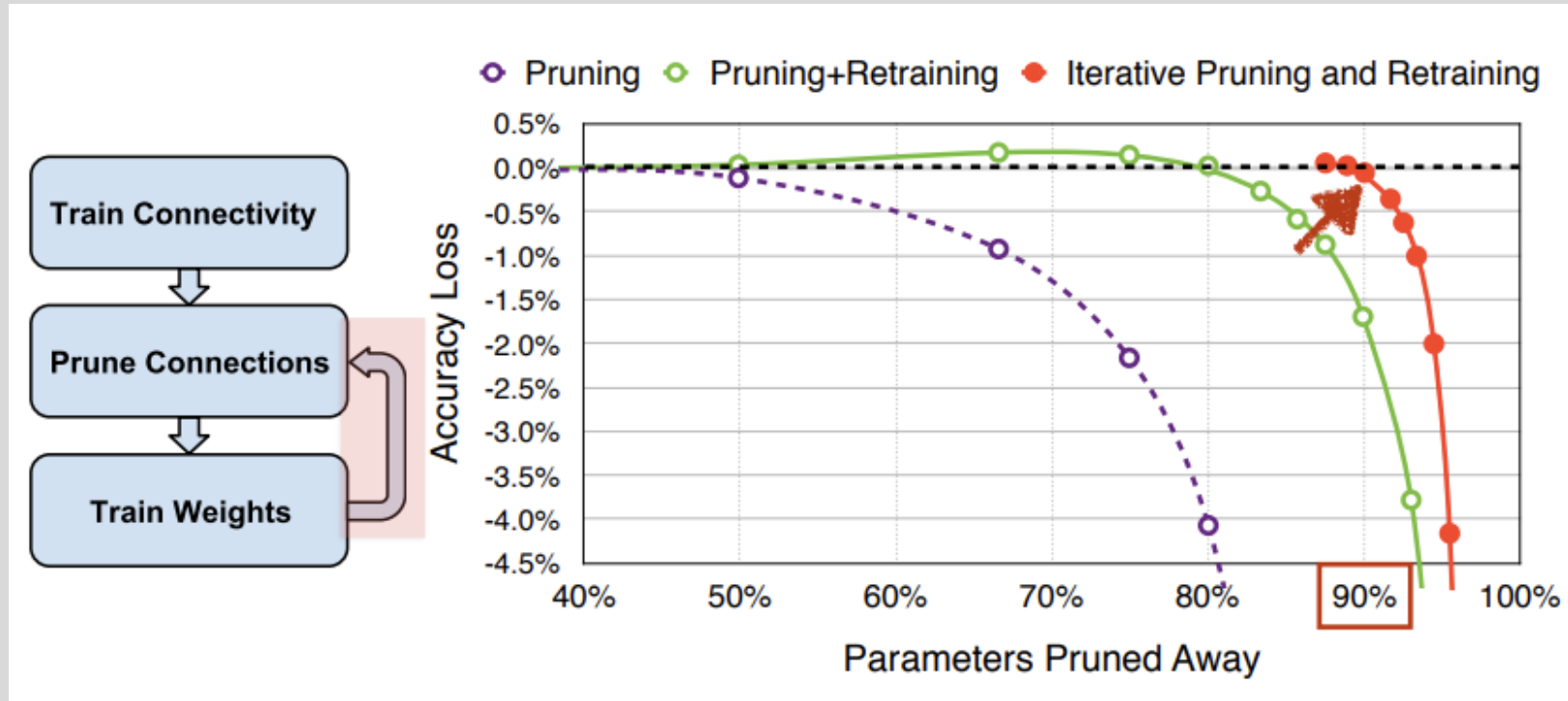
end for

Replay memory를 초기화 시키고 학습을 진행하며 낮은 확률로 정책을 따르지 않고 다른 행동을 하는 경우를 추가하고, 행동, 보상, 다음 상태를 확인하고, 그것을 replay memory에 저장하고, replay memory에서 샘플을 추출하여 기울기 하강을 수행한다.

Efficient method

모델의 필요한 연산량이 증가하여 효율적인 방법을 사용하여 학습시키거나 효율적인 하드웨어를 사용하는 것이 중요하다.

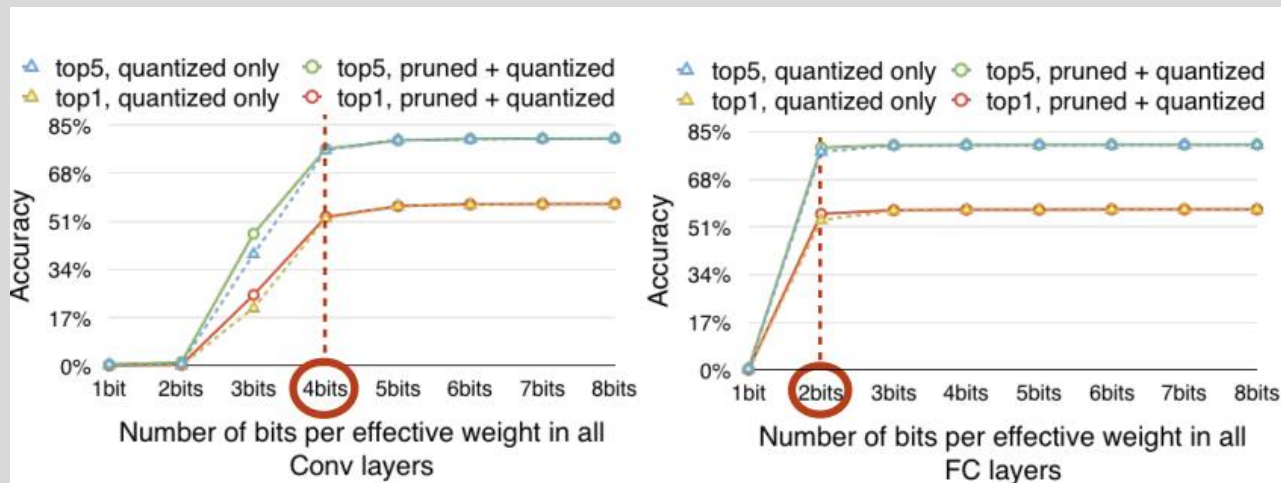
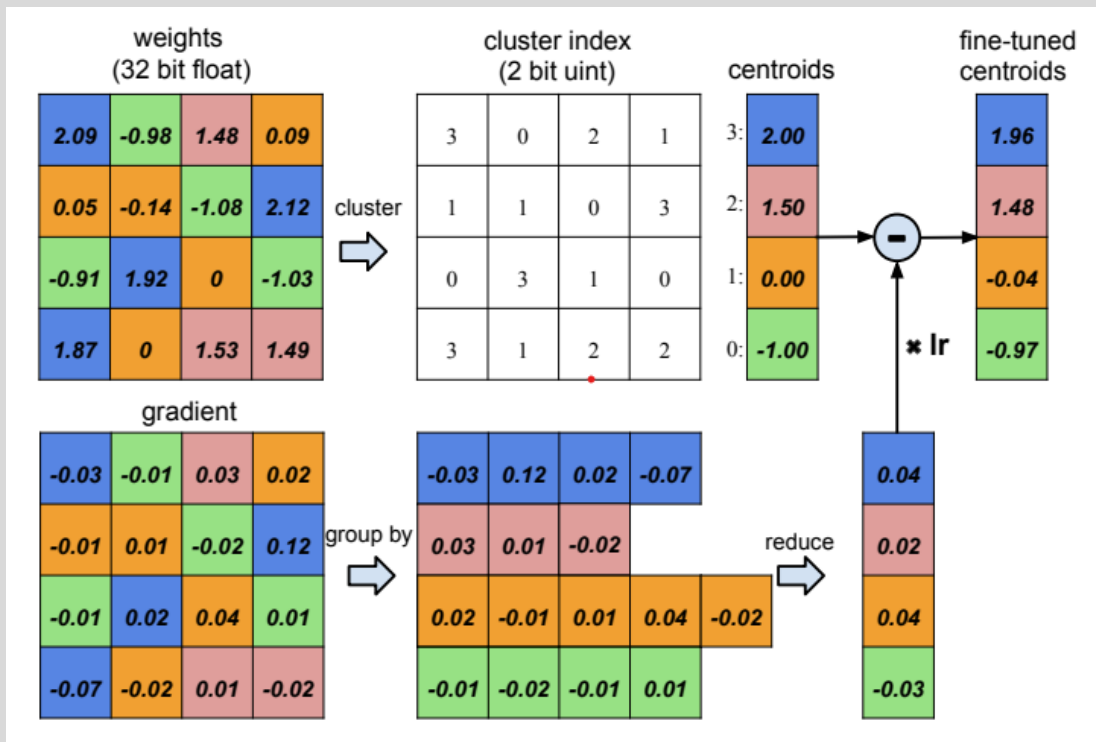
Pruning 학습에 가중치가 낮은 뉴런의 시냅스를 잘라내어 적은 뉴런들로 모델을 구성하여 연산량을 줄이는 방법



학습이 안정되면 불필요한 시냅스를 없애고 재학습하여 적은 뉴런으로 비슷한 성능을 보여줌

Efficient method

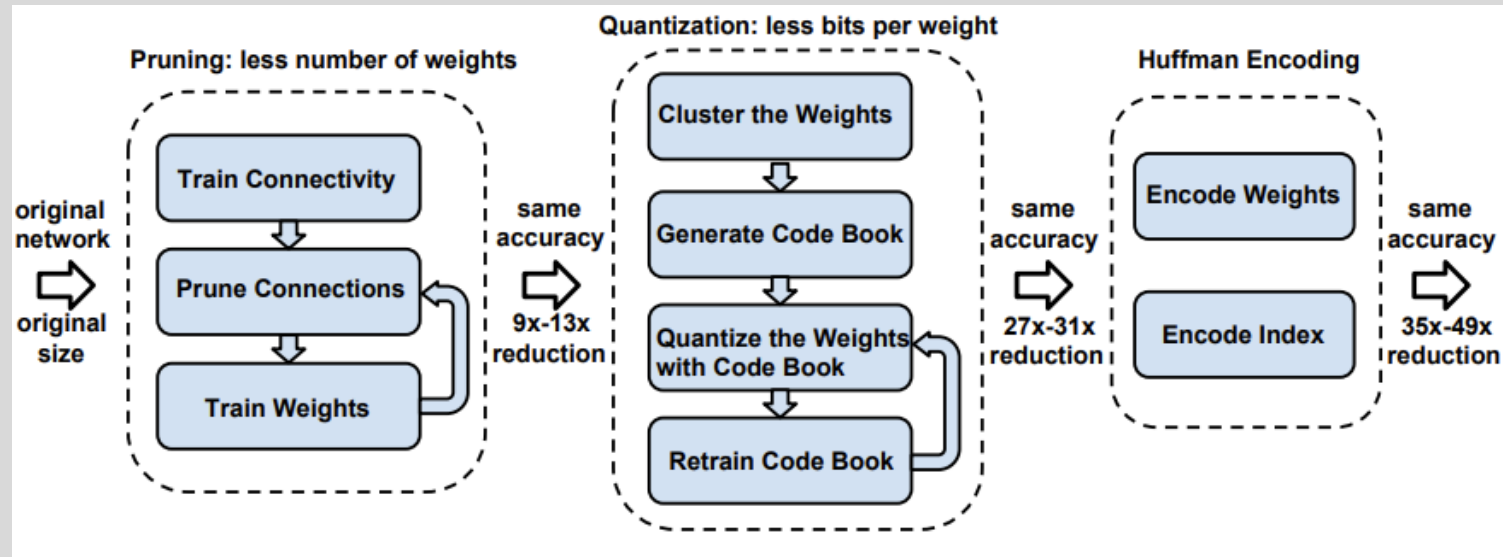
Trained Quantization은 숫자의 정밀도를 낮추어 비슷한 값끼리 묶어서 연산하는 방법



Quantize한 비트에 따른 정확도를 보면 Conv layer는 4비트
Fc layer는 2비트 부터 성능이 좋아짐

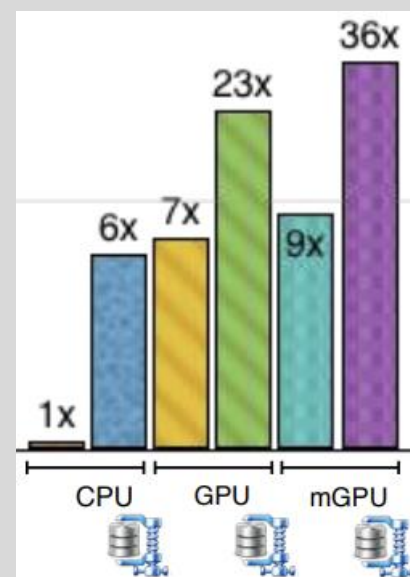
Huffman coding을 통해 많이 나오는 weight나 gradient의
경우 많은 비트를 주고 아닌 경우에는 적은 비트를 할당할 수 있
음

Efficient method

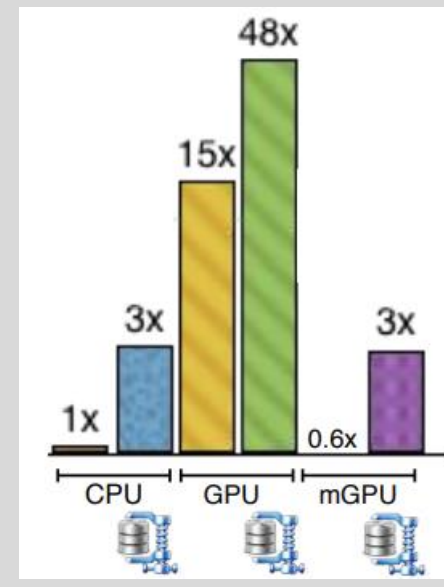


세개를 다 적용한 Deep Compression의 구조
속도와 에너지효율에서 3배가량 좋아짐

Network	Original Size	Compressed Size	Compression Ratio	Original Accuracy	Compressed Accuracy
LeNet-300	1070KB	→ 27KB	40x	98.36%	→ 98.42%
LeNet-5	1720KB	→ 44KB	39x	99.20%	→ 99.26%
AlexNet	240MB	→ 6.9MB	35x	80.27%	→ 80.30%
VGGNet	550MB	→ 11.3MB	49x	88.68%	→ 89.09%
GoogleNet	28MB	→ 2.8MB	10x	88.90%	→ 88.92%
ResNet-18	44.6MB	→ 4.0MB	11x	89.24%	→ 89.28%



메모리 효율

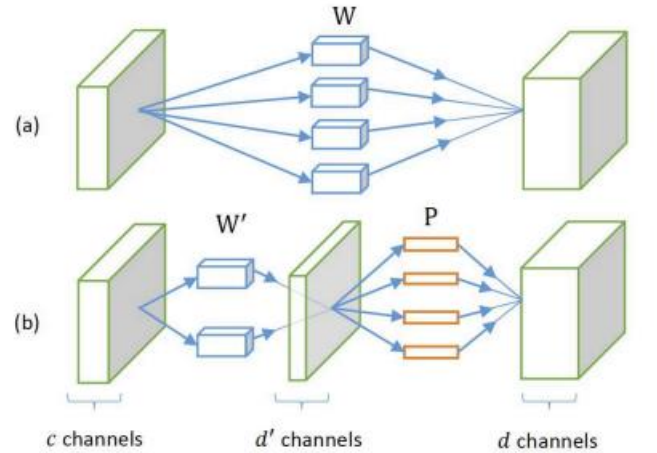


속도

Efficient method

Low Rank Approximation 저차원의 행렬 연산으로 바꿔 계산량을 줄인다

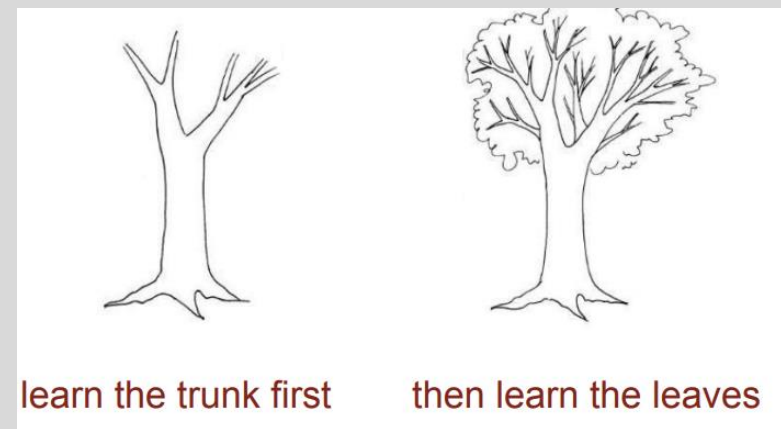
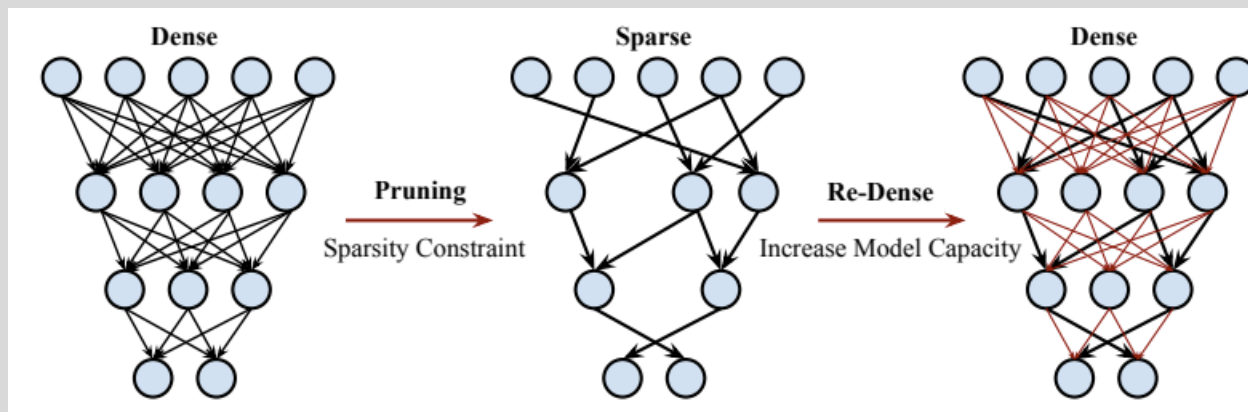
- Layer responses lie in a low-rank subspace
- Decompose a convolutional layer with d filters with filter size $k \times k \times c$ to
 - A layer with d' filters ($k \times k \times c$)
 - A layer with d filter ($1 \times 1 \times d'$)



Binary/Ternary Net 가중치와 activation 출력을 0과1 혹은 -1,0,1로 하여 효율성을 높이는 방법

Efficient method

DSD: pruning에서 끊어진 시냅스를 연결시켜 다시 dense한 모델로 학습시키는 방법

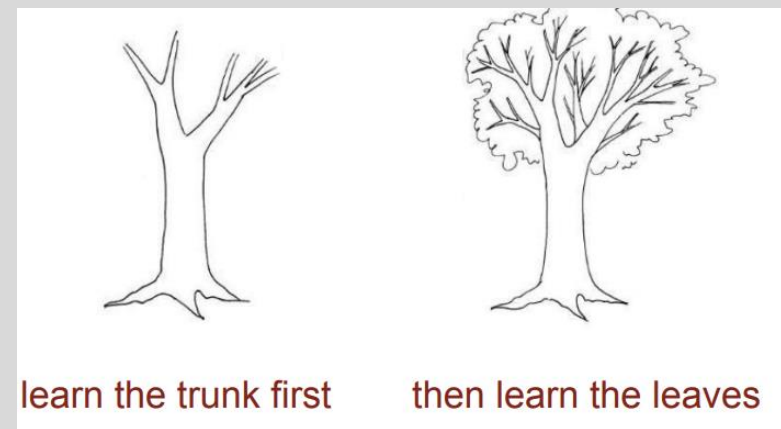
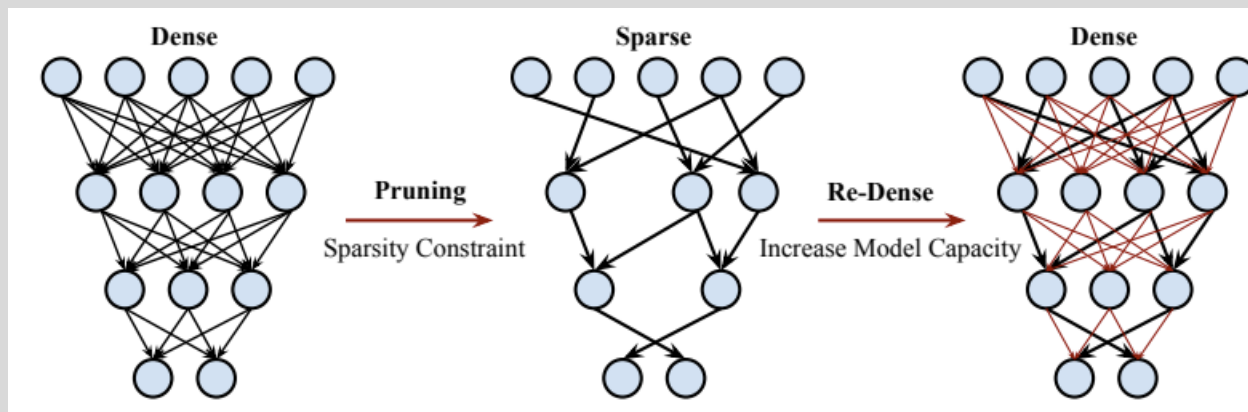


Baseline	Top-1 error	Top-5 error	DSD	Top-1 error	Top-5 error
AlexNet	42.78%	19.73%	AlexNet_DSD	41.48%	18.71%
VGG16	31.50%	11.32%	VGG16_DSD	27.19%	8.67%
GoogleNet	31.14%	10.96%	GoogleNet_DSD	30.02%	10.34%
SqueezeNet	42.39%	19.32%	SqueezeNet_DSD	38.24%	16.53%
ResNet18	30.43%	10.76%	ResNet18_DSD	29.17%	10.13%
ResNet50	24.01%	7.02%	ResNet50_DSD	22.89%	6.47%

효율적인 학습과 성능 향상을 보임

Efficient method

DSD: pruning에서 끊어진 시냅스를 연결시켜 다시 dense한 모델로 학습시키는 방법



Baseline	Top-1 error	Top-5 error	DSD	Top-1 error	Top-5 error
AlexNet	42.78%	19.73%	AlexNet_DSD	41.48%	18.71%
VGG16	31.50%	11.32%	VGG16_DSD	27.19%	8.67%
GoogleNet	31.14%	10.96%	GoogleNet_DSD	30.02%	10.34%
SqueezeNet	42.39%	19.32%	SqueezeNet_DSD	38.24%	16.53%
ResNet18	30.43%	10.76%	ResNet18_DSD	29.17%	10.13%
ResNet50	24.01%	7.02%	ResNet50_DSD	22.89%	6.47%

효율적인 학습과 성능 향상을 보임

GAN 코드 구현

Discriminator 구조

```
model = nn.Sequential(  
    nn.Flatten(),  
    nn.Linear(784, 256),  
    nn.LeakyReLU(0.01),  
    nn.Linear(256, 256),  
    nn.LeakyReLU(0.01),  
    nn.Linear(256, 1))
```

Generator 구조

```
model = nn.Sequential(nn.Linear(noise_dim, 1024),  
    nn.ReLU(),  
    nn.Linear(1024, 1024),  
    nn.ReLU(),  
    nn.Linear(1024, 784),  
    nn.Tanh())
```

Discriminator loss

```
logits_real = bce_loss(logits_real, torch.ones_like(logits_real).reshape(-1))  
logits_fake = bce_loss(logits_fake, torch.zeros_like(logits_fake).reshape(-1))  
loss = logits_real + logits_fake
```

Generator loss

```
loss = bce_loss(logits_fake, torch.ones_like(logits_fake).reshape(-1))
```

Optimizer

```
optimizer = optim.Adam(model.parameters(), lr=0.0001, betas=(0.5, 0.999))
```

MNIST 데이터셋
3750번째 생성 이미지



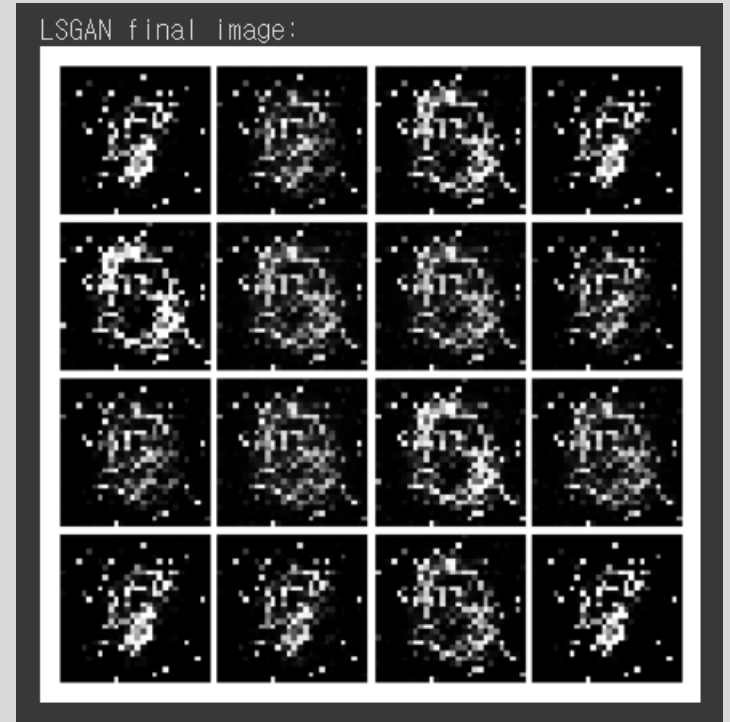
GAN 코드 구현

Least Square GAN모델을 구현하기위해 손실함수를 바꿈

$$\ell_G = \frac{1}{2} \mathbb{E}_{z \sim p(z)} \left[(D(G(z)) - 1)^2 \right]$$

$$\ell_D = \frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}} \left[(D(x) - 1)^2 \right] + \frac{1}{2} \mathbb{E}_{z \sim p(z)} \left[(D(G(z)))^2 \right]$$

생성한 이미지



GAN 코드 구현

Deep convolutional GAN 구현

Discriminator

```
model = nn.Sequential(nn.Conv2d(1, 32, kernel_size = 5, stride = 1),
    nn.LeakyReLU(0.01),
    nn.MaxPool2d(2,2),
    nn.Conv2d(32,64, kernel_size=5,stride=1),
    nn.LeakyReLU(0.01),
    nn.MaxPool2d(2,2),
    Flatten(),
    nn.Linear(4*4*64,4*4*64),
    nn.LeakyReLU(0.01),
    nn.Linear(4*4*64,1))
return model
```

Generator

```
model = nn.Sequential(
    nn.Linear(noise_dim,1024),
    nn.ReLU(),
    nn.BatchNorm1d(1024),
    nn.Linear(1024,7*7*128),
    nn.ReLU(),
    nn.BatchNorm1d(7*7*128),
    nn.Unflatten(1, (128,7,7)),
    nn.ConvTranspose2d(128,64,kernel_size=4,stride=2,padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(64),
    nn.ConvTranspose2d(64,1,kernel_size=4,stride=2,padding=1),
    nn.Tanh(),
    nn.Flatten()
)
return model
```

Iter: 1750



GAN 코드 구현

GAN의 train과정 discriminator의 학습후 generator의 학습 진행

```
images = []
iter_count = 0
for epoch in range(num_epochs):
    for x, _ in loader_train:
        if len(x) != batch_size:
            continue
        D_solver.zero_grad()
        real_data = x.type(dtype)
        logits_real = D(2* (real_data - 0.5)).type(dtype)

        g_fake_seed = sample_noise(batch_size, noise_size).type(dtype)
        fake_images = G(g_fake_seed).detach()
        logits_fake = D(fake_images.view(batch_size, 1, 28, 28))

        d_total_error = discriminator_loss(logits_real, logits_fake)
        d_total_error.backward()
        D_solver.step()

        G_solver.zero_grad()
        g_fake_seed = sample_noise(batch_size, noise_size).type(dtype)
        fake_images = G(g_fake_seed)

        gen_logits_fake = D(fake_images.view(batch_size, 1, 28, 28))
        g_error = generator_loss(gen_logits_fake)
        g_error.backward()
        G_solver.step()

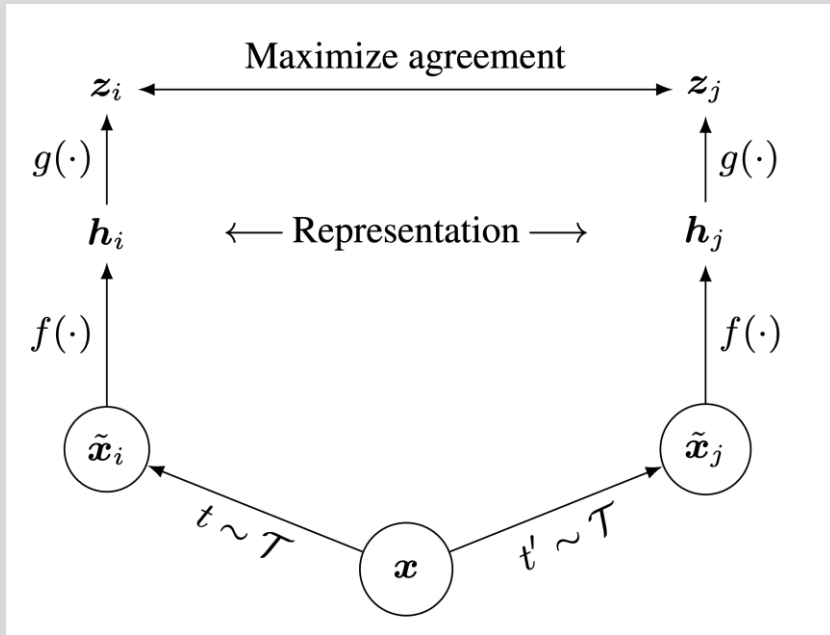
        if (iter_count % show_every == 0):
            print('Iter: {}, D: {:.4}, G:{:.4}'.format(iter_count,d_total_error.item(),g_error.item()))
            imgs_numpy = fake_images.data.cpu().numpy()
            images.append(imgs_numpy[0:16])

    iter_count += 1

return images
```

Self-supervised learning - SimCLR

Self Supervised Learning 구현 - SimCLR



SimCLR구조

같은 데이터를 다르게 변형시켜 두개를 나누고 $f()$ 를 거쳐 특징을 추출하고 $g()$ 를 거쳐 동일한 이미지를 판단하게 학습시켜

시각적 표현을 학습시키는 방법

Self-supervised learning - SimCLR

손실함수

$$l(i, j) = -\log \frac{\exp(\text{sim}(z_i, z_j) / \tau)}{\sum_{k=1}^{2N} 1_{k \neq i} \exp(\text{sim}(z_i, z_k) / \tau)}$$

$$\text{sim}(z_i, z_j) = \frac{z_i \cdot z_j}{\|z_i\| \|z_j\|}$$

$$L = \frac{1}{2N} \sum_{k=1}^N [l(k, k+N) + l(k+N, k)]$$

```
sims_k = torch.tensor([sim(z_k, z) for z in out[np.arange(2*N)!=k]])
sims_k_N = torch.tensor([sim(z_k_N, z) for z in out[np.arange(2*N)!=k+N]])

l1 = -((sim(z_k, z_k_N) / tau).exp() / (sims_k / tau).exp().sum()).log()
l2 = -((sim(z_k_N, z_k) / tau).exp() / (sims_k_N / tau).exp().sum()).log()

# Update the total loss
total_loss += l1 + l2
```

학습에서 사전 학습된 모델을 이용하고 분류학습을 하는 과정에서는 모델을 G()부분을 없애고 이전 파라미터를 고정시키고 classification layer를 추가하여 학습을 진행함

